

# □ Email Spam Detection Using Machine Learning

## Internship Project | Python + Scikit-Learn

**Goal:** Build a machine learning model that classifies emails as **Spam** or **Not Spam (Ham)**

**Libraries Used:** NumPy, Pandas, Matplotlib, Seaborn, Scikit-Learn

```
import numpy as np
import pandas as pd

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report
```

```
import pandas as pd
```

```
df = pd.read_csv(
    r"C:\Users\neelg\Downloads\spam.csv",
    encoding='latin1'
)
```

```
df.head()
```

|   | v1   | v2                                                | Unnamed: 2 |
|---|------|---------------------------------------------------|------------|
| 0 | ham  | Go until jurong point, crazy.. Available only ... | NaN        |
| 1 | ham  | Ok lar... Joking wif u oni...                     | NaN        |
| 2 | spam | Free entry in 2 a wkly comp to win FA Cup fina... | NaN        |
| 3 | ham  | U dun say so early hor... U c already then say... | NaN        |
| 4 | ham  | Nah I don't think he goes to usf, he lives aro... | NaN        |

|   | Unnamed: 3 | Unnamed: 4 |
|---|------------|------------|
| 0 | NaN        | NaN        |
| 1 | NaN        | NaN        |

|   |     |     |
|---|-----|-----|
| 2 | NaN | NaN |
| 3 | NaN | NaN |
| 4 | NaN | NaN |

## DATASET INFORMATION

```
print("Dataset shape", df.shape)
print("Dataset info:")
print(df.info())
```

Dataset shape (5572, 5)  
Dataset info:  
<class 'pandas.DataFrame'>  
RangeIndex: 5572 entries, 0 to 5571  
Data columns (total 5 columns):

| # | Column     | Non-Null Count | Dtype |
|---|------------|----------------|-------|
| 0 | v1         | 5572 non-null  | str   |
| 1 | v2         | 5572 non-null  | str   |
| 2 | Unnamed: 2 | 50 non-null    | str   |
| 3 | Unnamed: 3 | 12 non-null    | str   |
| 4 | Unnamed: 4 | 6 non-null     | str   |

dtypes: str(5)  
memory usage: 217.8 KB  
None

## checking missing values

```
print("Missing values in Dataset")
print(df.isnull().sum())
```

Missing values in Dataset

|            |      |
|------------|------|
| v1         | 0    |
| v2         | 0    |
| Unnamed: 2 | 5522 |
| Unnamed: 3 | 5560 |
| Unnamed: 4 | 5566 |

dtype: int64

INSIGHTS: There are no Missing values in dataset

```
print(df.columns)
df = df[['v1', 'v2']].copy()

df.columns = ['Category', 'Message']

df.head()
```

```
Index(['v1', 'v2', 'Unnamed: 2', 'Unnamed: 3', 'Unnamed: 4'],
      dtype='str')
```

|   | Category | Message                                           |
|---|----------|---------------------------------------------------|
| 0 | ham      | Go until jurong point, crazy.. Available only ... |
| 1 | ham      | Ok lar... Joking wif u oni...                     |
| 2 | spam     | Free entry in 2 a wkly comp to win FA Cup fina... |
| 3 | ham      | U dun say so early hor... U c already then say... |
| 4 | ham      | Nah I don't think he goes to usf, he lives aro... |

## CONVERT LABEL TO NUMBERS

```
df['Category'].value_counts()
```

```
Category
ham      4825
spam      747
Name: count, dtype: int64
```

```
df['Category'] = df['Category'].map({
    'ham':0,
    'spam':1
})
```

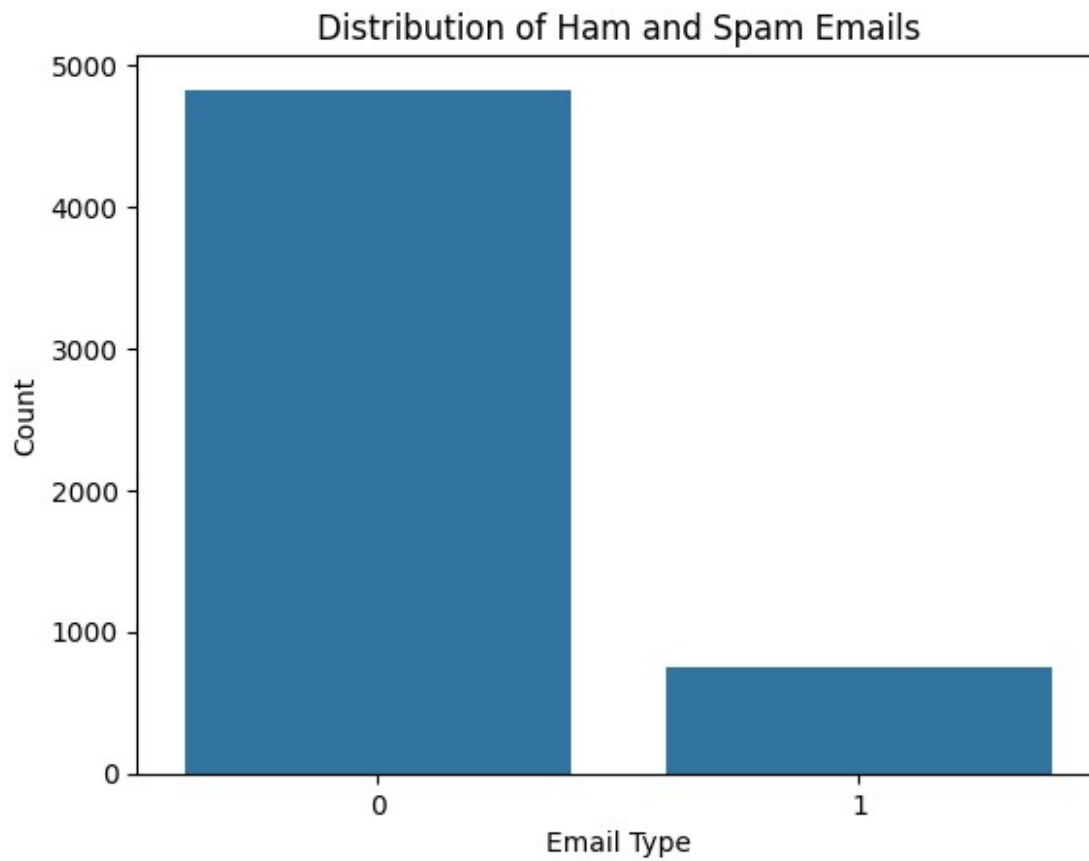
```
df
```

|      | Category | Message                                           |
|------|----------|---------------------------------------------------|
| 0    | 0        | Go until jurong point, crazy.. Available only ... |
| 1    | 0        | Ok lar... Joking wif u oni...                     |
| 2    | 1        | Free entry in 2 a wkly comp to win FA Cup fina... |
| 3    | 0        | U dun say so early hor... U c already then say... |
| 4    | 0        | Nah I don't think he goes to usf, he lives aro... |
| ...  | ...      | ...                                               |
| 5567 | 1        | This is the 2nd time we have tried 2 contact u... |
| 5568 | 0        | Will I_b going to esplanade fr home?              |
| 5569 | 0        | Pity, * was in mood for that. So...any other s... |
| 5570 | 0        | The guy did some bitching but I acted like i'd... |
| 5571 | 0        | Rofl. Its true to its name                        |

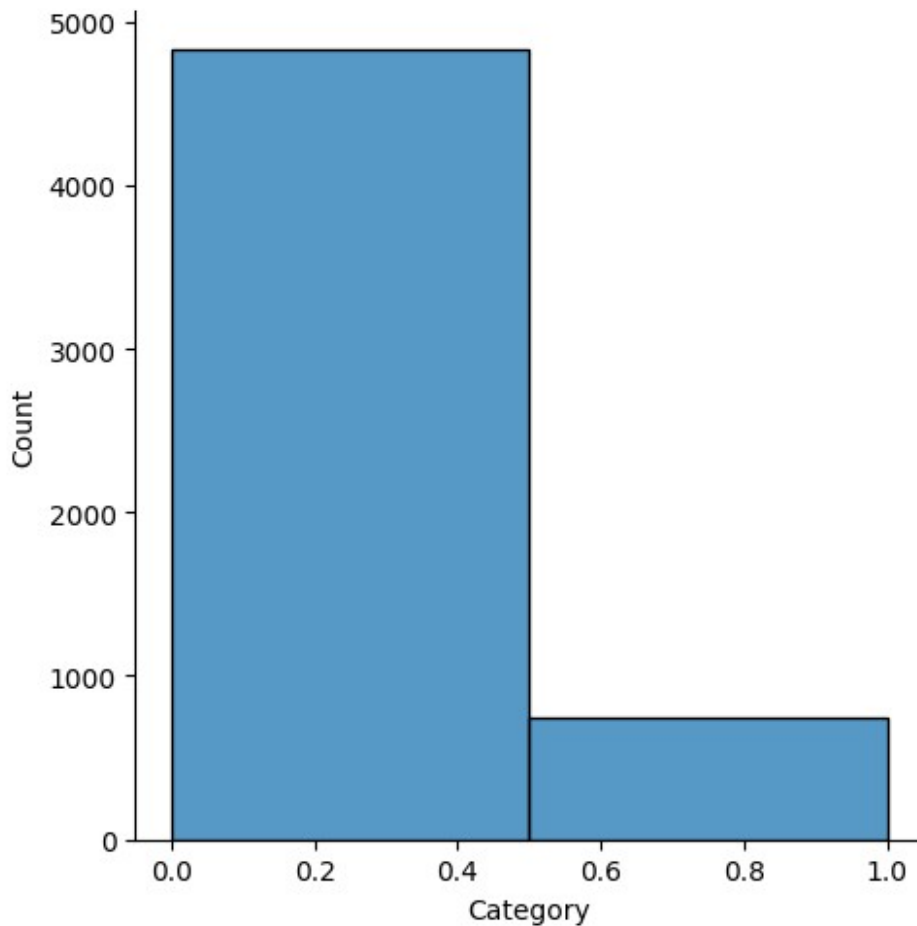
```
[5572 rows x 2 columns]
```

## VISULIZE SPAM AND HAM

```
sns.countplot(x='Category', data=df)
plt.title("Distribution of Ham and Spam Emails")
plt.xlabel("Email Type")
plt.ylabel("Count")
plt.show()
```



```
sns.displot(data=df, x='Category', kind='hist',bins=2)  
<seaborn.axisgrid.FacetGrid at 0x16e8d128ad0>
```



```
X=df['Category']  
y=df['Message']
```

CONVERT TEXT INTO NUMBERS

```
X = df['Message']      # Email text  
y = df['Category']     # Spam/Ham label  
  
# Features and Labels  
X = df['Message'].astype(str)  
y = df['Category']  
  
# Convert text into numbers  
from sklearn.feature_extraction.text import TfidfVectorizer  
  
vectorizer = TfidfVectorizer()  
  
X = vectorizer.fit_transform(X)  
  
print("Transformation Successful")  
print(X.shape)
```

Transformation Successful  
(5572, 8672)

TRAIN TEST SPLIT

```
X_train, X_test, y_train, y_test = train_test_split(X,y, test_size =  
0.2, random_state = 42)
```

MODEL TRAINING AND PREDICTION

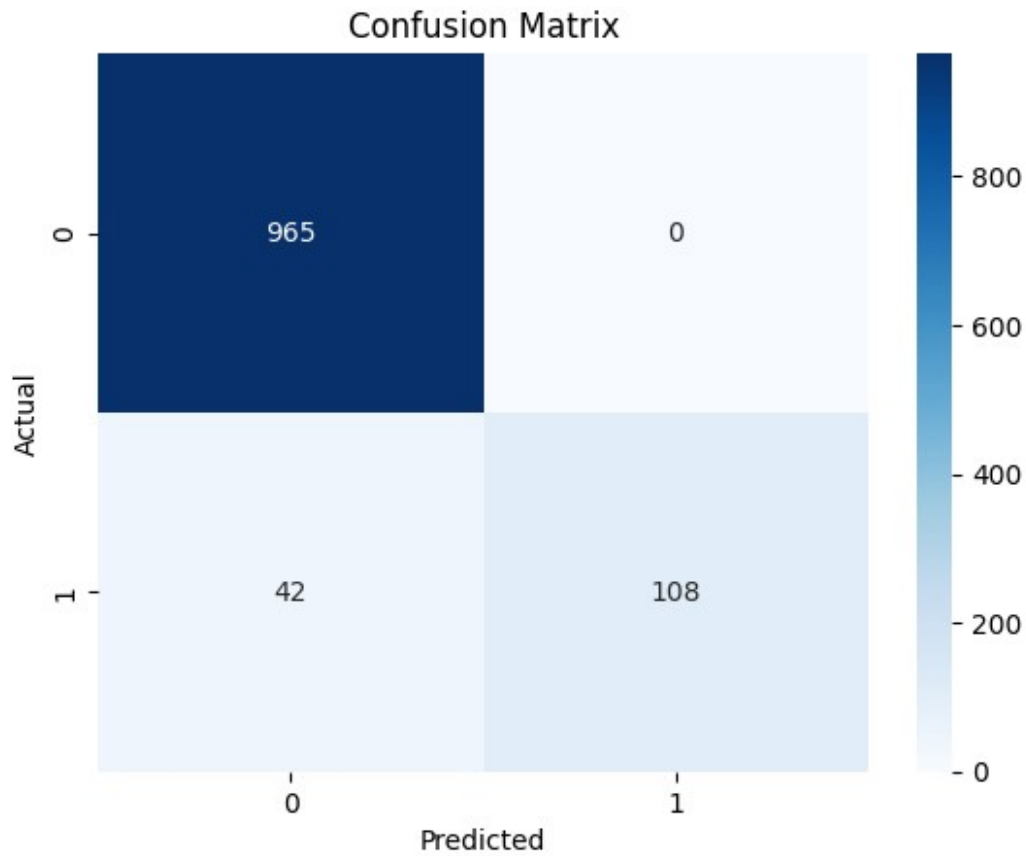
```
model = MultinomialNB()  
model.fit(X_train, y_train)  
  
y_pred= model.predict(X_test)
```

MODEL ACCURACY

```
accuracy = accuracy_score(y_test,y_pred)  
print("Accuracy",accuracy*100,"%")  
  
Accuracy 96.23318385650225 %
```

CONFUSION METRIX

```
cm = confusion_matrix(y_test, y_pred)  
  
sns.heatmap(  
    cm,  
    annot=True,  
    fmt='d',  
    cmap='Blues'  
)  
  
plt.xlabel("Predicted")  
plt.ylabel("Actual")  
plt.title("Confusion Matrix")  
  
plt.show()
```



#### CLASSIFICATION REPORT

```
print(classification_report(y_test, y_pred))
```

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.96      | 1.00   | 0.98     | 965     |
| 1            | 1.00      | 0.72   | 0.84     | 150     |
| accuracy     |           |        | 0.96     | 1115    |
| macro avg    | 0.98      | 0.86   | 0.91     | 1115    |
| weighted avg | 0.96      | 0.96   | 0.96     | 1115    |

testing on email

```
email = ["Congratulations! You have won a free iPhone. Click now!"]
email_vector = vectorizer.transform(email)
prediction = model.predict(email_vector)
if prediction[0] == 1:
```

```
    print("Spam Email")  
else:  
    print("Not Spam Email")
```

Spam Email